SC3270 Reasoning About Programs
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-30 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1 What Correct Means and Hoare Triples **1**

1.1 Why Testing Is Not Enough . . . 1

1.2 States, Assertions, Triples . . . 2

1.3 A First Proof Without Machinery . . . 3

1.4 Consequence: Strengthening and Weakening . . . 3

1.5 Worked Example: Specifying Maximum . . . 4

1.6 What Partial Leaves Out: Preview of Termination . . . 4

1.7 Exercises . . . 5

2 Preconditions and Postconditions: Writing Contracts **6**

2.1 Contracts Before Code . . . 6

2.2 Strength: Weaker and Stronger . . . 7

2.3 Writing Good Specs and Spotting Bad Ones . . . 8

2.4 Quantifiers, English, and Class Invariants . . . 9

2.5 Exercises . . . 9

3 Invariants: What Stays True **11**

3.1 What Is an Invariant? . . . 11

3.2 Discovering Invariants: Done vs. Remaining . . . 12

3.3 Too Weak, Too Strong, Just Right . . . 12

3.4 Why Invariants Suffice: Induction Over Iterations . . . 13

3.5 Inventing vs. Checking . . . 14

3.6 Worked Mini-Proof: Summation End-to-End . . . 14

3.7 Exercises . . . 14

4 Verifying Loops: Putting Invariants to Work **16**

4.1 The While-Rule and Its Three Obligations . . . 16

4.2 Warm-Up: Summation Revisited as a VC Proof . . . 17

4.3 Search and Euclid: Invariants That Carry Quantifiers . . . 18

4.4 Nested Loops and Auxiliary Variables . . . 19

4.5 Generating VCs Mechanically . . . 19

4.6 Case Study: Verifying Partition . . . 19

4.7 Exercises . . . 20

5 Termination: Proving Programs Stop **21**

5.1 Partial Is Not Enough . . . 21

5.2 Variants: Numbers That Must Hit Zero . . . 22

5.3 Proving Total Correctness . . . 23

5.4 When Termination Fails . . . 23

5.5 Well-Founded Orders Beyond $\mathbb{N}$ . . . 24

5.6 Connecting to Decidability . . . 24

5.7 Full Total-Correctness Proof: Linear Search . . . 24

5.8 Exercises . . . 25

6 Weakest Preconditions: Reasoning Backwards **26**

6.1 Reasoning Backwards . . . 26

6.2 Calculus: wp by Syntax . . . 27

6.3 Loops: Approximation and Invariants . . . 28

6.4 wp and Symbolic Execution . . . 29

6.5 Dual: Strongest Postconditions . . . 29

6.6 Worked End-to-End: Swap . . . 29

6.7 Exercises . . . 30

7 Separation Logic: Reasoning About Heap and Pointers **31**

7.1 Why Hoare Logic Stumbles on the Heap . . . 31

7.2 Heaps, Points-To, and Separating Conjunction . . . 32

7.3 Heap Axioms and the Frame Rule . . . 33
7.4 Lists and Local Reversal . . . 34
7.5 Magic Wand and Iterated Star . . . 34
7.6 Why Frame Prevents Leaks and Double-Free . . . 34
7.7 Exercises . . . 35

8 Refinement and Correctness by Construction 36

8.1 From Post-Hoc Proof to Construction . . . 36
8.2 Laws of Refinement . . . 37
8.3 Construction in Action: Binary Search . . . 38
8.4 Heap Refinement: List Reversal Redux . . . 39
8.5 Mini-Derivation: Summation Lawfully . . . 39
8.6 When to Refine vs Verify . . . 39
8.7 Exercises . . . 39

Chapter 1

What Correct Means and Hoare Triples

> *"A program is correct not when it passes ten tests, but when no test could make it fail."* We move from sampling behaviours (testing) to quantifying over all behaviours (proof).

From zero: no verification background assumed. We define what a program does via its states, build up what an assertion means, and introduce Hoare triples as contracts. Pictures come first; every formal rule is motivated by a tiny program you can run. Only basic programming and high-school logic are assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define program state and assertion as a set of states;

(L2) write a Hoare triple $\{P\}\, C\, \{Q\}$ and read its meaning;

(L3) distinguish partial correctness from total correctness;

(L4) explain why testing can miss bugs that proof must cover;

(L5) state the intuitive meaning of consequence and composition.

1.1 Why Testing Is Not Enough

Intuition. Think of a program as a machine that reads a suitcase of variables $\{x, y, \dots\}$ and transforms it. The suitcase contents is the *state*. Testing opens the machine on ten suitcases and checks the outputs. Proof asks: does the machine do the right thing on *every* suitcase that satisfies the boarding condition?

Consider absolute value:

```
if x < 0 then x := -x
```

Testing $x = 2$ and $x = -3$ succeeds. Yet $x = -2^{31}$ on 32-bit two's complement overflows. A proof quantifies over all x.

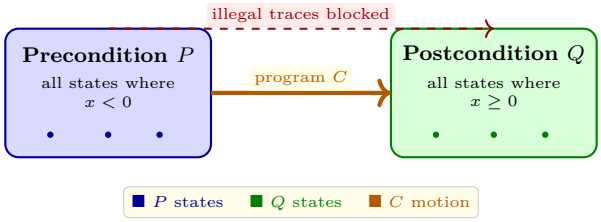

Figure 1.1: A Hoare triple $\{P\}\, C\, \{Q\}$: every P-state moved by C must land in Q. The program is a conveyor from the blue set to the green set.

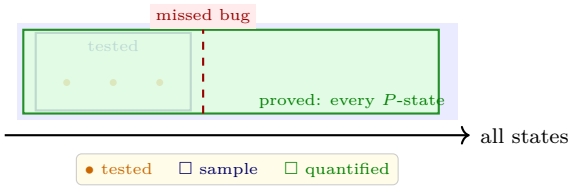

Figure 1.2: Testing samples finitely many points (orange dots) inside the blue sample box; proof shades the entire green region. A bug lurking outside the sample is invisible to tests but caught by proof.

1.2 States, Assertions, Triples

Definition 1.1 (State and assertion)**.** A *state* σ is a mapping from variables to values, e.g. $\{x \mapsto 3, y \mapsto -1\}$. An *assertion* P is a logical formula over program variables; it denotes the *set* of states where P holds: $[\![P]\!] = \{\sigma \mid \sigma \models P\}$.

So $x > 0$ is not a boolean computed by the program—it is a *description* of infinitely many states. Writing $x = y + 1$ means the set where the equation holds.

Definition 1.2 (Hoare triple)**.** $\{P\}\, C\, \{Q\}$ means: if C starts in any state satisfying P and C terminates, then the final state satisfies Q. This is *partial* correctness—it says nothing if C loops forever. *Total* correctness $[P]\, C\, [Q]$ additionally requires termination.

Example: $\{\text{true}\}\ x := x + 1\ \{x > 0\}$ is false (start $x = -5$, end -4). $\{x > 0\}\ x := x + 1\ \{x > 1\}$ is true. Formally, $\{P\}\, C\, \{Q\}$ is a specification; C is an implementation. The triple is the *conveyor* picture: P states ride C and must arrive in Q.

Listing 1.1: Assertions as runtime checks: the idea behind Hoare triples made executable.

```
# Assertion as set of states: we test membership by evaluating a formula
def triple_holds(P, C, Q, states):
    """Check {P} C {Q} by brute force on a finite sample of states."""
    for s in states:
        if P(s):                # only P-states matter
            s2 = C(s.copy())    # run C
            if s2 is not None and not Q(s2):
                return False, s, s2  # counterexample: P held, Q failed
    return True, None, None
```

```python

# Example: absolute value program
def C_abs(s):
    if s['x'] < 0: s['x'] = -s['x']
    return s

P = lambda s: True              # any starting state
Q = lambda s: s['x'] >= 0       # must be non-negative afterwards
print(triple_holds(P, C_abs, Q, [{'x':v} for v in [-3,0,2,5]]))
```

1.3 A First Proof Without Machinery

Take $C \equiv x := x+1;\ y := 2*x$ with spec $\{x = 3\}\ C\ \{y = 8\}$. Informally: after $x := x+1$, we know $x = 4$; then $y := 2 * x$ gives $y = 8$. That two-step chain already uses the deepest Hoare ideas: assignment as substitution and sequencing as composition. We will formalise them as axioms in Chapter 2; for now see the pattern.

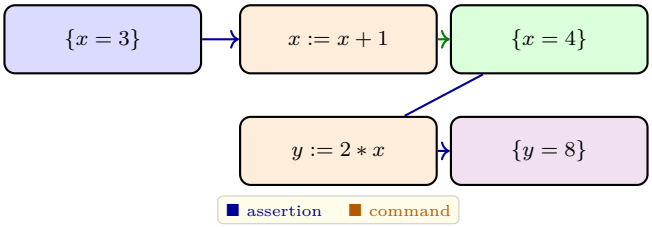

Figure 1.3: Sequencing as a conveyor: the intermediate assertion $\{x = 4\}$ is the handoff point between two assignments.

Listing 1.2: Same idea in C: pre/post as comments. The proof obligation is that no P-state escapes to $\neg Q$.

```c
// { x == 3 }    <-- P holds before
x = x + 1;       // now { x == 4 } must hold (assignment axiom preview)
// { x == 4 }
y = 2 * x;       // now { y == 8 } must hold
// { y == 8 }    <-- Q holds after, if we terminate
```

Partial vs. total: $\{x \geq 0\}$ `while` $x > 0$ `do` $x := x - 1$ $\{x = 0\}$ is true for partial correctness (if it stops, $x = 0$). It is also totally correct because the loop always stops—Chapter 6 will prove it. But $\{\text{true}\}$ `while true do skip` $\{\text{false}\}$ is vacuously partially correct (it never terminates, so the "if it terminates" is empty), yet not totally correct.

Remark 1.1 (Vacuous truth). $\{\text{false}\}\,C\,\{Q\}$ is always true: there is no P-state to check. This mirrors "if false then anything" and explains why an impossible precondition makes any spec trivially hold—watch for it when you strengthen preconditions too far.

1.4 Consequence: Strengthening and Weakening

Intuition. If you prove $\{P\}\,C\,\{Q\}$ and you know $P' \Rightarrow P$ (so P' is smaller, stronger), then $\{P'\}\,C\,\{Q\}$ is free: every P'-state is already a P-state. Dually, if $Q \Rightarrow Q'$ (so Q' is larger, weaker), then $\{P\}\,C\,\{Q'\}$ follows—landing in Q guarantees landing in the bigger set Q'.

Think of sets. Let $[\![P']\!] \subseteq [\![P]\!]$ and $[\![Q]\!] \subseteq [\![Q']\!]$. The triple says the image of $[\![P]\!]$ under C sits inside $[\![Q]\!]$. Then the image of the smaller $[\![P']\!]$ sits inside $[\![Q]\!] \subseteq [\![Q']\!]$. No re-proof of C needed.

Formally this is the rule of *consequence*; Chapter 2 makes it an inference rule:

$$\frac{P' \Rightarrow P \quad \{P\}\, C\, \{Q\} \quad Q \Rightarrow Q'}{\{P'\}\, C\, \{Q'\}}.$$

It is the workhorse for fitting a proven triple to the pre/post your caller actually has.

Example 1.1 (Consequence in use)**.** We prove $\{x = 3\}\, x := x + 1\, \{x = 4\}$. A caller has $\{x = 3 \wedge y = 0\}$. Since $(x = 3 \wedge y = 0) \Rightarrow (x = 3)$, consequence gives $\{x = 3 \wedge y = 0\}\, x := x + 1\, \{x = 4\}$. Then weaken $x = 4 \Rightarrow x > 0$ to obtain $\{x = 3 \wedge y = 0\}\, x := x + 1\, \{x > 0\}$. The program did not change; our description did.

1.5 Worked Example: Specifying Maximum

Specify $m :=$ if $a \geq b$ then a else b so that m is the maximum. Attempt 1: $\{\text{true}\}\, C\, \{m \geq a \wedge m \geq b\}$ is true but too weak—it allows $m = 100$ when $a = 2, b = 3$. We need "m equals one of the inputs and dominates both":

$$\{\text{true}\}\ C\ \{(m = a \vee m = b)\ \wedge\ m \geq a\ \wedge\ m \geq b\}.$$

Equivalently $m = \max(a, b)$. The disjunction is essential because we do not know statically which branch runs.

Why not $\{a \geq b\}\, C\, \{m = a\}$? That is true but incomplete—it only handles half the inputs. The full spec with true as pre forces the program to handle both branches, and the proof must case-split on $a \geq b$.

This pattern recurs: a precise postcondition often needs $\vee$, $\exists$, or quantifiers to describe "whichever path was taken, the output has this shape." Vague posts give vacuous proofs.

Remark 1.2 (Contracts as promises). A triple is a *contract*: the caller promises to establish P; the callee promises to establish Q if it terminates. This is identical to JML's `requires/ensures` or Eiffel's `pre/post` that you will write in Chapter 3. The proof is the evidence that the promise is kept for all admissible callers.

1.6 What Partial Leaves Out: Preview of Termination

Partial correctness guarantees "if you finish, you finish right" but never promises finishing. The classic villain is a loop that never makes progress:

`while x > 0 do x := x` — x never changes, so $\{x \geq 0\}\, C\, \{x = 0\}$ is partially correct (vacuously when $x > 0$, because it never exits), yet totally false.

Dijkstra called this the "fire alarm" problem: a program that burns forever still satisfies every partial spec. Chapter 6 adds *variants* to rule it out. For now remember: total correctness = partial + termination, and the two concerns need different proof tools (invariants vs. ranking functions).

Example 1.2 (Absolute value total)**.** $\{\text{true}\}$ `if` $x < 0$ `then` $x := -x$ $\{x \geq 0\}$ is both partially and totally correct: the program has no loops, so termination is trivial. Contrast $\{\text{true}\}$ `while` $x < 0$ `do` $x := x + 1$ $\{x \geq$

0}—partially correct (if it stops, $x \geq 0$), but totally correct only when x is bounded above; we must exhibit a variant $-x$ that decreases.

1.7 Exercises

E1.1 **State sets.** Write the set $[\![x > 0 \wedge y = x + 1]\!]$ informally. Give two states inside and two outside. Why is an assertion a set, not a single state?

E1.2 **True or false triples.** Decide for each: (a) $\{x = 5\}\, x := x + 1\, \{x = 6\}$, (b) $\{x > 0\}\, x := x - 1\, \{x \geq 0\}$, (c) $\{\text{true}\}$ `while true do skip` $\{\text{false}\}$ for partial correctness. Justify.

E1.3 **Max spec.** Write a Hoare triple for $m := \max(a, b)$ using $\{\text{true}\}$ as pre and a post with disjunction. Why is disjunction needed?

E1.4 **Testing blind spot.** Give a program where testing 100 random states all satisfy $\{P\}\, C\, \{Q\}$ yet the triple is false. Exhibit the counterexample state and explain why random sampling missed it logically.

E1.5 **Partial vs total.** State $\{n \geq 0\}$ `while` $n > 0$ `do` $n := n - 1\, \{n = 0\}$ in both partial and total form. Which extra obligation does total add, and why is it non-trivial?

Chapter Notes

Hoare's 1969 paper "An Axiomatic Basis for Computer Programming" introduced triples. Partial vs. total correctness is Floyd/Hoare; Manna–Waldinger and Winskel give gentle introductions. The conveyor picture is standard in Pierce's *Software Foundations*, Vol. 1, Ch. Hoare. For a historical view see Jones, "The Early Search for Tractable Ways of Reasoning About Programs."

Remark 1.3 (How to read the rest of this book). Each chapter follows intuition → formal → example → exercise. Run every listing, draw every diagram on paper, and instantiate each triple with two concrete states—one that should be accepted and one rejected. That habit turns abstract logic into debugging skill.

Chapter 2

Preconditions and Postconditions: Writing Contracts

> *"A function is a contract: if you give me this, I promise that."* Preconditions are what the caller must achieve; postconditions are what the callee guarantees in return.

From zero: no contract experience assumed. We build contracts from English to logic, show why strong and weak matter, and prove that precondition-strengthening and postcondition-weakening are safe. Every notion is illustrated on tiny programs before formal rules.

Learning Outcomes

After this chapter you will be able to:

(L1) formalise informal requirements as pre/post assertions with quantifiers;

(L2) compare contract strength and order them by implication;

(L3) apply precondition strengthening and postcondition weakening soundly;

(L4) detect vacuous, too-weak, and too-strong specifications;

(L5) write JML-style `requires/ensures` and class invariants.

2.1 Contracts Before Code

Intuition. A vending machine says "insert 1.20 €, press B3, receive cola." The *requires* is "you inserted 1.20 € and B3 is in stock"; the *ensures* is "you receive cola and change is correct." If you violate requires (insert 0.50 €), the machine may do anything—keep money, return error—no guarantee.

A program function is identical:

$$\texttt{requires } P \quad \texttt{function } f(x) \quad \texttt{ensures } Q$$

The caller must make P true before the call; the function must make Q true after, provided it terminates. The Hoare triple $\{P\}\, f\, \{Q\}$ is exactly this contract.

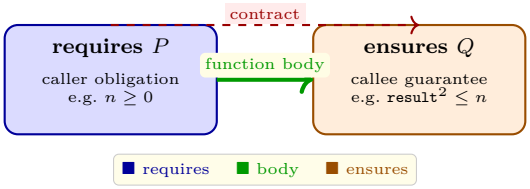

Figure 2.1: A contract as a bridge: the caller builds P, the body carries across, the caller may assume Q. Violate P and the bridge has no deck.

Definition 2.1 (Contract)**.** A *contract* for command C is a pair (P, Q) of assertions. C *satisfies* it iff $\{P\}\, C\, \{Q\}$ holds. P is the *precondition*, Q the *postcondition*. When C is a function with parameters $\bar{x}$ and result r, we write free variables accordingly.

Example 2.1 (Square root spec)**.** Informal: "$\texttt{isqrt}(n)$ returns the integer square root of n." Formal:

$$\{n \geq 0\} \;\; r := \texttt{isqrt}(n) \;\; \{r \geq 0 \;\wedge\; r^2 \leq n \;<\; (r+1)^2\}.$$

Without $n \geq 0$, no integer r satisfies the post when $n = -1$—the spec would be unsatisfiable and any implementation vacuously "correct" on negative inputs, hiding the bug that caller and callee disagree on domain.

2.2 Strength: Weaker and Stronger

Assertions ordered by $\Rightarrow$ form a lattice. Recall: P is *stronger* than P' if $P \Rightarrow P'$ (fewer states satisfy it). P is *weaker* if $P' \Rightarrow P$. Strong = small set, weak = large set.

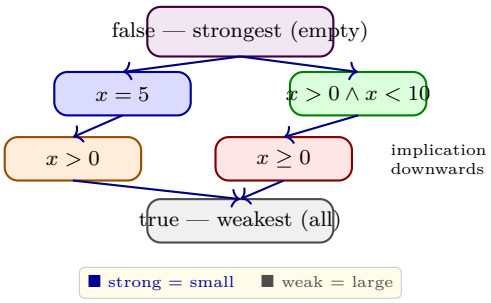

Figure 2.2: Strength lattice: implication flows downward; stronger means fewer states. Moving up shrinks, moving down grows.

Why care? A caller wants a *weak* pre (easy to satisfy) and a *strong* post (much information). A callee wants the opposite: strong pre (assume more) and weak post (promise less). Good specification is a negotiation. The two safe moves are:

Theorem 2.1 (Consequence moves)**.** If $\{P\}\, C\, \{Q\}$ and $P' \Rightarrow P$ then $\{P'\}\, C\, \{Q\}$ (strengthen pre). If $Q \Rightarrow Q'$ then $\{P\}\, C\, \{Q'\}$ (weaken post). Both are sound.

So you may always *strengthen* the requires and *weaken* the ensures without re-proving the body. The opposite directions are unsound: weakening pre admits new starting states the body was never verified for; strengthening post claims more than was proved.

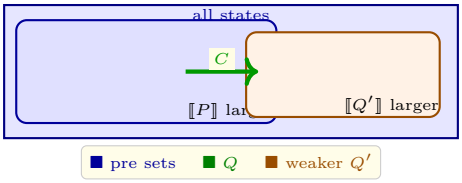

Figure 2.3: Safe moves: shrinking the pre (stronger P') and growing the post (weaker Q') preserve the triple. The image of P' under C still lands inside $Q \subseteq Q'$.

2.3 Writing Good Specs and Spotting Bad Ones

Three prototypical bad specs:

Vacuous: $\{\text{false}\}\, C\, \{Q\}$—always true, tells nothing. Symptom: pre contains contradiction like $x > 0 \wedge x < 0$. Fix: weaken pre until satisfiable.

Too weak post: $\{\text{true}\}\, r := \texttt{isqrt}(n)\, \{r \geq 0\}$—true even for $r = 999$. Fix: strengthen post until it pins down the desired answer.

Too strong pre: $\{\text{sorted}(a) \wedge n = 2^k\}\, \texttt{bsearch}\, \{\text{found}\}$—excludes most arrays. Fix: weaken pre to the minimal assumption the algorithm actually needs.

Example 2.2 (Binary search contract)**.** Good: $\{\text{sorted}(a[0..n-1]) \wedge 0 \leq n\}\ r := \texttt{bsearch}(a, n, key)\ \{(r \geq 0 \Rightarrow a[r] = key) \wedge (r = -1 \Rightarrow key \notin a)\}$. Every conjunct is load-bearing: sorted is needed for correctness, the two implications capture both outcomes precisely.

Listing 2.1: JML-style contract for integer square root: the spec the code must satisfy.

```
/* @ requires n >= 0;
  @ ensures \result >= 0 && \result*\result <= n
  @       && n < (\result+1)*(\result+1);
  @*/
int isqrt(int n){
    int r = 0;
    while((r+1)*(r+1) <= n) r++;
    return r;
}
```

Listing 2.2: Checking strength: implication as subset test on samples.

```
def implies(P, Q, domain):
    """Does P => Q hold on finite domain sample?"""
    for s in domain:
        if P(s) and not Q(s):
            return False, s  # s satisfies P but not Q
    return True, None

```

```
P  = lambda s: s['x']==5
Q1 = lambda s: s['x']>0
Q2 = lambda s: s['x']>10
print(implies(P, Q1, [{'x':i} for i in range(-2,12)])) # True: 5=> >0
print(implies(P, Q2, [{'x':i} for i in range(-2,12)])) # False: counterexample x=5
```

2.4 Quantifiers, English, and Class Invariants

Specs of collections need quantifiers. "a is sorted" abbreviates $\forall i.\, 0 \le i < n-1 \Rightarrow a[i] \le a[i+1]$. "$key$ appears at k" is $\exists k.\, 0 \le k < n \wedge a[k] = key$. Translating English is error-prone: "all evens are positive" is $\forall x.\, (even(x) \Rightarrow x > 0)$, not $\forall x.\, even(x) \wedge x > 0$ (which claims everything is even!).

A *class invariant* is a condition that every public method may assume on entry and must re-establish on exit. For a sorted list object:

$$\texttt{inv}: \; 0 \le \texttt{size} \le \texttt{capacity} \;\wedge\; \text{sorted}(a[0..\texttt{size}-1]).$$

The invariant is conjoined to every method's pre and post; it is the "steady truth" between calls. Chapter 4 will elevate this to loop invariants—the same idea, but preserved across iterations rather than method calls.

Example 2.3 (Quantifier pitfalls)**.** Spec for "r is first occurrence of key":

$$\{\text{true}\}\; r := \texttt{first}(a, key)\; \{(r = -1 \Rightarrow \forall i.\, a[i] \neq key) \;\wedge\; (r \ge 0 \Rightarrow a[r] = key \wedge \forall i < r.\, a[i] \neq key)\}.$$

Dropping $\forall i < r$ gives a post that allows a later occurrence to be returned—too weak. Adding $\forall i.\, a[i] \neq key$ to the second conjunct makes it unsatisfiable when key exists—too strong.

2.5 Exercises

E2.1 **Strength ordering.** Order by strength: $x = 3$, $x \ge 3$, $x > 0$, true, false, $x \ge 0 \wedge x \le 5$. Draw the implication lattice.

E2.2 **Strengthen/Weaken soundness.** Give a concrete C, P, P' where $P \Rightarrow P'$ but $\{P\}\, C\, \{Q\}$ holds while $\{P'\}\, C\, \{Q\}$ fails. Explain why weakening pre is unsafe.

E2.3 **Sorted contract.** Write pre/post for linear search without assuming sortedness but guaranteeing the same two-case post as binary search. Compare strength of the two pres.

E2.4 **Spot the vacuous.** Find the vacuous triple among: (a) $\{x > 0 \wedge x < 0\}\, x := 1\, \{x = 1\}$, (b) $\{x \ge 0\}\, x := x + 1\, \{x > 0\}$, (c) $\{\text{true}\}\, \texttt{skip}\, \{\text{false}\}$. For the vacuous one, give the weakest satisfiable strengthening of pre that makes it meaningful.

E2.5 **JML translation.** Translate the informal "$\texttt{divide}(a, b)$ returns q with $a = bq + r$ and $0 \le r < b$" into a formal pre/post with quantifiers. What happens if you drop $b > 0$ from pre?

Chapter Notes

Design by Contract originates with Meyer's Eiffel; JML (Leavens et al.) brings it to Java. The lattice view of assertions is standard in abstract interpretation (Cousot). For practice, write contracts before code—specification is the hardest and most valuable part of verification.

Remark 2.1 (Checklist for a good contract). Ask: is P satisfiable? Is Q strong enough to be useful? Does Q avoid claiming the unclaimable (no disjunction forgotten, no $\forall$ where $\exists$ needed)? If you cannot state Q precisely, you do not yet understand what the program should do.

Chapter 3

Invariants: What Stays True

> *"An invariant is the sentence a loop keeps repeating to itself so it does not get lost."* Find that sentence and you can prove the loop correct without unrolling it infinitely.

From zero: no invariant experience assumed. We define invariants from scratch, show how to discover them by splitting what is done from what remains, and practise judging too-weak and too-strong candidates. Pictures guide every definition; formalism follows.

Learning Outcomes

After this chapter you will be able to:

(L1) define invariant (loop, data, class) and state its preservation condition;

(L2) propose an invariant for a simple loop by done/remaining decomposition;

(L3) check that invariant $\wedge$ $\neg$guard implies postcondition;

(L4) diagnose invariants that are too weak or too strong;

(L5) explain why invariants are the induction hypothesis of a loop.

3.1 What Is an Invariant?

Intuition. Watch someone sorting a hand of cards by insertion: after each insertion, the left portion is sorted. That sortedness never breaks—it is *invariant* across iterations. If you know what stays sorted and what remains unsorted, you understand the whole algorithm without watching every swap.

A loop invariant is a *property of the state* that is true before the loop, stays true after each iteration, and—together with the loop having exited—implies the desired postcondition. Three obligations:

$$\text{(init) } P \Rightarrow I \qquad \text{(preserve) } \{I \wedge B\}\ C\ \{I\} \qquad \text{(exit) } I \wedge \neg B \Rightarrow Q.$$

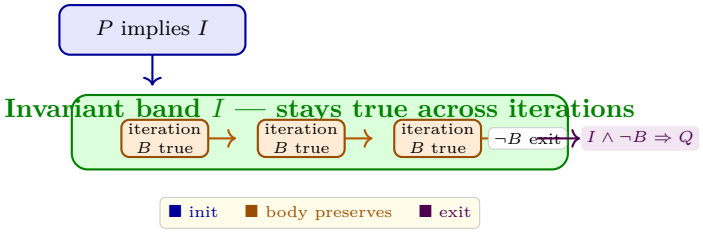

Figure 3.1: Invariant as a fence: you enter inside I (init), each lap stays inside (preserve), and leaving through the $\neg B$ gate lands in Q.

Like induction, the invariant is the *induction hypothesis* for loops: base case is $P \Rightarrow I$, inductive step is $\{I \wedge B\}\, C\, \{I\}$, conclusion is $I \wedge \neg B \Rightarrow Q$.

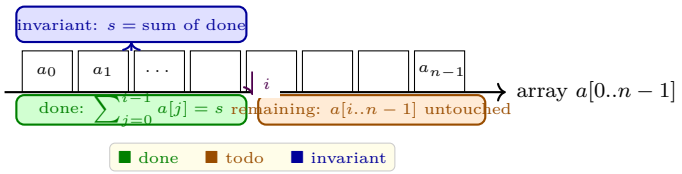

Figure 3.2: Discovery pattern for summation: $I \equiv 0 \le i \le n \wedge s = \sum_{j=0}^{i-1} a[j]$. Done plus todo equals whole job.

3.2 Discovering Invariants: Done vs. Remaining

A reliable recipe: ask *what has the loop achieved so far?* and *what is left?* The invariant ties the two to the final goal.

Summation: $s := 0$; $i := 0$; `while` $i < n$ `do` $s := s + a[i]$; $i := i + 1$ with $\{\text{true}\}\ \ldots\ \{s = \sum_{j=0}^{n-1} a[j]\}$. Done $= \sum_{j<i}$, remaining $= \sum_{j \ge i}$. So propose $I \equiv 0 \le i \le n \wedge s = \sum_{j=0}^{i-1} a[j]$. Check: init $i = 0 \Rightarrow s = 0 = \sum_{\emptyset} = 0$. Preserve: adding $a[i]$ shifts done by one. Exit $i = n \Rightarrow s = \sum_{j=0}^{n-1} a[j] = Q$.

Listing 3.1: Summation with invariant as executable assertion: fails loudly if discovery is wrong.

```
def sum_array(a):
    s, i, n = 0, 0, len(a)
    # I: 0<=i<=n and s == sum(a[0:i])
    assert 0 <= i <= n and s == sum(a[0:i])
    while i < n:
        # I and B (i<n) hold
        s += a[i]; i += 1
        assert 0 <= i <= n and s == sum(a[0:i])  # I preserved
    # I and not B (i==n) => s == sum(a)
    assert s == sum(a)
    return s
```

Counting positives: $c := 0$; $i := 0$; `while` $i < n$ `do if` $a[i] > 0$ `then` $c := c + 1$; $i := i + 1$ yields $I \equiv 0 \le i \le n \wedge c = |\{j < i : a[j] > 0\}|$.

3.3 Too Weak, Too Strong, Just Right

An invariant can fail two ways:

Too weak: $\{\text{true}\}\, C\, \{Q\}$ holds for any C but does not imply Q at exit. Example: $I \equiv 0 \le i \le n$ for summation—preserved, but $I \wedge i = n$ says only $i = n$, not s's value. So $I \wedge \neg B \not\Rightarrow Q$.

Too strong: $I \equiv s = \sum_{j=0}^{i-1} a[j] \wedge i = n$ claims the loop is already done before it starts—init fails because $0 = n$ is false unless $n = 0$.

Goldilocks I is strong enough to imply Q at exit, weak enough to be established initially and preserved. Strengthening a weak I (add the s conjunct) or weakening a strong I (drop $i = n$) moves toward the sweet spot.

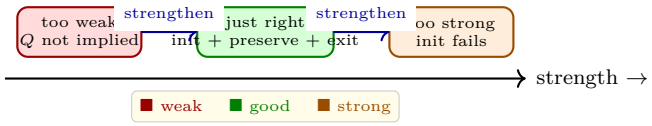

Figure 3.3: Invariant strength spectrum: weak cannot reach Q, strong cannot be entered, just-right does both.

Listing 3.2: Three candidates for summation—only one survives all three checks.

```
// Candidate A (too weak):  0<=i<=n
//   preserve OK, but upon exit i==n does NOT imply s==sum(a)

// Candidate B (just right): 0<=i<=n && s==sum(a[0..i-1])
//   init: i=0,s=0 OK; preserve: s+=a[i],i++ OK; exit: i==n => s==sum(a)

// Candidate C (too strong): s==sum(a[0..i-1]) && i==n
//   init fails when n>0: i==n is false at start
```

Example 3.1 (Linear search invariant)**.** $a[0..n-1]$, search key, $i := 0$; `while` $i < n \wedge a[i] \neq key$ `do` $i := i + 1$. Good invariant: $I \equiv 0 \le i \le n \wedge (\forall j < i.\, a[j] \neq key)$. Then $I \wedge \neg(i < n \wedge a[i] \neq key)$ splits into $i = n \vee a[i] = key$, and I tells us no earlier position held key—exactly the post we want.

3.4 Why Invariants Suffice: Induction Over Iterations

Unrolling a loop yields a sequence of states $\sigma_0, \sigma_1, \ldots$ where σ_{k+1} is after $k+1$ iterations. The preservation condition $\{I \wedge B\}\, C\, \{I\}$ says: if $\sigma_k \models I \wedge B$ then $\sigma_{k+1} \models I$. By induction on k, every reachable $\sigma_k \models I$. So I holds no matter how many times we go around. At exit we additionally know $\neg B$, and $I \wedge \neg B \Rightarrow Q$ transfers truth to the post.

This is why invariants beat testing: testing checks finitely many σ_k; induction covers *all* k, including the largest n that triggers overflow, aliasing, or off-by-one that a test suite likely misses.

Example 3.2 (Off-by-one caught)**.** Program: $i := 0$; $s := 0$; `while` $i < n$ `do` $s := s + a[i]$; $i := i + 1$ but writer forgets $i := i+1$. Invariant $I \equiv 0 \le i \le n \wedge s = \sum_{j<i} a[j]$ is not preserved: after one body execution i unchanged, so s becomes $s + a[i]$ while i still old, breaking $s = \sum_{j<i} a[j]$. The preservation VC $I \wedge i < n \Rightarrow \text{wp}(s := s + a[i];\, I)$ fails—proof catches the missing increment statically.

Remark 3.1 (Data invariants vs loop invariants). A *data invariant* holds between method calls (e.g., "list is sorted"); a *loop invariant* holds between iterations. Both are preserved by a step, but scope differs. A class invariant broken inside a method is fine provided it is re-established before return, just as a loop invariant may be broken mid-body provided it is restored by the iteration's end.

3.5 Inventing vs. Checking

Finding I is creative; checking it is mechanical. In modern verifiers (Dafny, Why3) you *supply* I and the tool generates VCs $P \Rightarrow I$, $I \wedge B \Rightarrow \mathrm{wp}(C, I)$, $I \wedge \neg B \Rightarrow Q$ and discharges them with an SMT solver. So skill splits: human invents the shape (done/remaining), machine checks the details.

Heuristic checklist: (1) include bounds $0 \le i \le n$—almost always needed; (2) state what done means with quantifiers; (3) ensure I mentions every variable the loop mutates; (4) test I on $i = 0$ (init) and $i = n$ (exit) before proving preserve.

3.6 Worked Mini-Proof: Summation End-to-End

Claim: $\{n \ge 0\}\ s := 0; i := 0;$ `while` $i < n$ `do` $(s := s + a[i]; i := i + 1)\ \{s = \sum_{j=0}^{n-1} a[j]\}$.

Take $I \equiv 0 \le i \le n \wedge s = \sum_{j=0}^{i-1} a[j]$. Show:

Init: From $s = 0, i = 0$ we have $0 \le 0 \le n$ and $\sum_{j=0}^{-1} a[j] = 0$ (empty sum), so I.

Preserve: Assume $I \wedge i < n$, i.e. $0 \le i < n \wedge s = \sum_{j=0}^{i-1} a[j]$. After $s' := s + a[i]$ we have $s' = \sum_{j=0}^{i} a[j]$; after $i' := i + 1$ we have $s' = \sum_{j=0}^{i'-1} a[j]$ and $0 \le i' \le n$. So I restored.

Exit: $I \wedge i \ge n$ gives $i = n$ (from $0 \le i \le n$) and thus $s = \sum_{j=0}^{n-1} a[j] = Q$.

Every step uses only substitution and arithmetic; no loop unrolling needed. That is the power: one invariant replaces infinitely many iteration cases. The same template works for product, count, and search—only the done conjunct changes.

Remark 3.2 (Empty sum convention). Define $\sum_{j=0}^{-1} a[j] = 0$ so the base case $i = 0$ holds uniformly. For product use 1; for $\forall j < i.\, P(j)$ use true when $i = 0$. Consistent base values make $P \Rightarrow I$ trivial.

3.7 Exercises

E3.1 **Propose and test.** Give I for counting zeros in $a[0..n-1]$ and check the three conditions $P \Rightarrow I$, $\{I \wedge B\}\, C\, \{I\}$, $I \wedge \neg B \Rightarrow Q$.

E3.2 **Too weak diagnosis.** Show $I \equiv 0 \le i \le n$ for the positivity counter is preserved but fails to imply $c = |\{j < n : a[j] > 0\}|$ at exit. What minimal conjunct fixes it?

E3.3 **Too strong diagnosis.** Show $I \equiv c = |\{j < i : a[j] > 0\}| \wedge i = n$ fails at init when $n > 0$. Weaken it minimally to a correct invariant.

E3.4 **Search invariant.** Prove that $I \equiv \forall j < i.\, a[j] \ne key$ for linear search implies the post $(r = -1 \Rightarrow \forall j.\, a[j] \ne key) \wedge (r \ge 0 \Rightarrow a[r] = key)$.

E3.5 **Done/remaining for product.** Write I for $p := \prod_{j=0}^{i-1} a[j]$ and verify preserve step uses the substitution axiom for $p := p * a[i]$.

Chapter Notes

The done/remaining heuristic is folklore in Gries, *The Science of Programming* (1981). Invariants as induction hypotheses appear in Floyd (1967) and Hoare (1969). For practice, annotate every loop you write this week with its invariant—even when not required to prove it.

Remark 3.3 (Mental trick). Before coding a loop, write I and Q first, then derive the guard as "not yet Q" and the body as "make progress toward Q while preserving I". This is correctness by construction in miniature—Chapter 8 scales it to whole programs.

Chapter 4

Verifying Loops: Putting Invariants to Work

"A loop without an invariant is a story without a thesis: it may be entertaining, but you cannot tell whether it ends correctly." This chapter turns invariants into complete proofs and verification conditions.

From zero: loop proofs built from first principles. We assume only the invariant idea from Chapter 3 and build the full Hoare while-rule, generate VCs mechanically, and verify classic loops end-to-end. No prior VC experience needed.

Learning Outcomes

After this chapter you will be able to:

(L1) state the Hoare while-rule with invariant annotation and its three VCs;

(L2) verify summation, search, and Euclid loops from invariant to post;

(L3) handle nested loops by conjoining invariants;

(L4) generate VCs from an annotated program systematically;

(L5) explain why auxiliary variables are sometimes needed.

4.1 The While-Rule and Its Three Obligations

Intuition. A loop `while` B `do` C repeatedly checks B; if true, runs C and repeats. The invariant I is the *loop's contract* with its surroundings. Annotate the loop as `while` B `do` $\{I\}\, C$. Then correctness reduces to three logical implications (VCs):

1. **Initiation:** $P \Rightarrow I$ — you enter the loop inside I.
2. **Preservation:** $\{I \wedge B\}\, C\, \{I\}$ — each lap stays inside I.
3. **Exit:** $I \wedge \neg B \Rightarrow Q$ — leaving through $\neg B$ lands in Q.

Formally:

$$\frac{\{I \wedge B\}\, C\, \{I\}}{\{I\}\, \texttt{while}\ B\ \texttt{do}\ C\, \{I \wedge \neg B\}} \qquad \text{and then consequence to fit } P, Q.$$

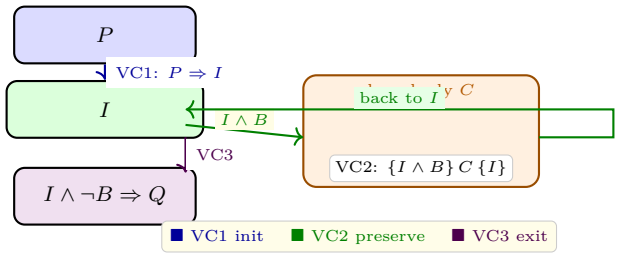

Figure 4.1: The three VCs of the while-rule: enter inside I, stay inside each lap, and exit into Q. If all three hold, the loop is partially correct.

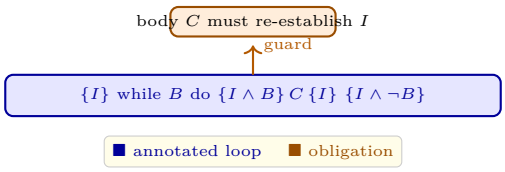

Figure 4.2: Annotated loop: the invariant appears explicitly as $\{I\}$ at the head; the body proof may assume $I \wedge B$ and must show I.

4.2 Warm-Up: Summation Revisited as a VC Proof

Program $S \equiv s := 0;\ i := 0;\ \texttt{while}\ i < n\ \texttt{do}\ (s := s + a[i];\ i := i + 1)$ with spec $\{n \geq 0\}\, S\, \{s = \sum_{j=0}^{n-1} a[j]\}$.

Pick $I \equiv 0 \leq i \leq n \wedge s = \sum_{j=0}^{i-1} a[j]$ (Chapter 3). VCs:

VC1: $n \geq 0 \wedge s = 0 \wedge i = 0 \Rightarrow I$ holds (empty sum).

VC2: $\{I \wedge i < n\}\, s := s + a[i];\ i := i + 1\, \{I\}$. After assignments, $s' = \sum_{j=0}^{i-1} a[j] + a[i] = \sum_{j=0}^{i} a[j]$ and $i' = i + 1$, so $s' = \sum_{j=0}^{i'-1} a[j]$.

VC3: $I \wedge i \geq n \Rightarrow i = n \Rightarrow s = \sum_{j=0}^{n-1} a[j]$.

All three are pure arithmetic—no program reasoning remains. That is the point: VCs are logic formulas an SMT solver can discharge. The human contribution ends at I; the solver handles the rest by unfolding definitions.

Remark 4.1 (Automation boundary). Writing I is the *specification* task; discharging VCs is the *verification* task. Tools like Dafny ask you for the former and automate the latter. Hence learning to craft I is not automatable away—it is the core intellectual skill of this course.

Listing 4.1: VCs as Python assertions: each VC is a logical check, not a test of runs.

```
# VC1: initiation
```

```
def vc1(n,s,i): return not (n>=0 and s==0 and i==0) or (0<=i<=n and s==sum([]))
# VC2: preservation (encoded as wp of body, Ch 7)
def vc2_body(a,i,s,n):
    # assume I and i<n; after s+=a[i]; i+=1, does I hold?
    I_before = (0<=i<=n and s==sum(a[0:i]))
    if not (I_before and i<n): return True
    s2, i2 = s+a[i], i+1
    return 0<=i2<=n and s2==sum(a[0:i2])
# VC3: exit
def vc3(a,s,i,n):
    I = (0<=i<=n and s==sum(a[0:i]))
    return not (I and not (i<n)) or (s==sum(a))
```

4.3 Search and Euclid: Invariants That Carry Quantifiers

Linear search: $i := 0$; `while` $i < n \wedge a[i] \neq key$ `do` $i := i + 1$ with post $(i = n \Rightarrow \forall j < n.\, a[j] \neq key) \wedge (i < n \Rightarrow a[i] = key)$. Invariant: $I \equiv 0 \leq i \leq n \wedge \forall j < i.\, a[j] \neq key$. VC2 uses the fact that if $\forall j < i.\, a[j] \neq key$ and $a[i] \neq key$ then $\forall j < i + 1.\, a[j] \neq key$.

Euclid GCD: $a, b > 0$; `while` $a \neq b$ `do if` $a > b$ `then` $a := a - b$ `else` $b := b - a$ with post $a = \gcd(a_0, b_0)$ where a_0, b_0 are initial values (ghost variables). Invariant: $I \equiv a > 0 \wedge b > 0 \wedge \gcd(a, b) = \gcd(a_0, b_0)$. Preservation uses $\gcd(a - b, b) = \gcd(a, b)$ when $a > b$. Exit $a = b$ gives $a = \gcd(a_0, b_0)$.

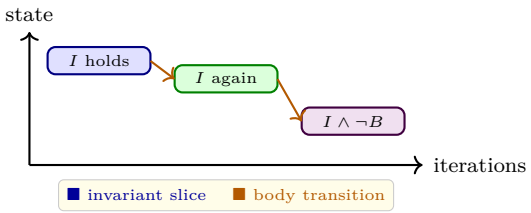

Figure 4.3: Euclid trace: each iteration preserves $I \equiv \gcd(a, b) = \gcd(a_0, b_0)$; exit with $a = b$ forces the post. The invariant is a coloured bar that never changes height.

Listing 4.2: Euclid with ghost variables to state the invariant: a_0, b_0 never change but appear in I.

```
def gcd_verify(a,b):
    a0,b0 = a,b
    # I: a>0 and b>0 and gcd(a,b)==gcd(a0,b0)
    import math
    assert a>0 and b>0 and math.gcd(a,b)==math.gcd(a0,b0)
    while a != b:
        if a > b: a = a - b
        else:     b = b - a
        assert a>0 and b>0 and math.gcd(a,b)==math.gcd(a0,b0)
    assert a == math.gcd(a0,b0)  # I and not B => post
    return a
```

4.4 Nested Loops and Auxiliary Variables

Nested loops conjoin invariants: outer I_{out} and inner I_{in} with I_{out} preserved across the whole inner loop as if it were one command. Example: sum of matrix $m \times n$: outer i has $I_{out} \equiv \sum_{\text{rows } <i}$, inner j has $I_{in} \equiv I_{out} \wedge \sum_{\text{cols } <j \text{ in row } i}$. Inner exit re-establishes I_{out} for next outer iteration.

Sometimes I needs an *auxiliary (ghost) variable* not in the program to remember history. For GCD we used a_0, b_0; for "x is the number of times B was true" we add counter t that shadows the loop. Auxiliary variables are removed before execution—they are proof scaffolding.

4.5 Generating VCs Mechanically

Given annotated program, VC generation is syntax-directed: walk the tree, emit implications. For sequence $C_1; C_2$, compose VCs; for conditional, split on guard; for while, emit the three VCs above. Tools like Why3 do this and feed formulas to Z3/CVC5. Your job as human: supply I; the rest is automatic.

Remark 4.2 (Annotation burden). The only creative input is the invariant. Everything else—assignment axiom $Q[x \mapsto e]$, sequence, conditional, consequence—is deterministic. This is why loop verification is "find I, then win".

4.6 Case Study: Verifying Partition

Partition (Lomuto) $i := -1$; `for` $j = 0$ `to` $n-1$ `do if` $a[j] \le pivot$ `then` $i := i+1$; $swap(a[i], a[j])$ with post $a[0..i] \le pivot < a[i+1..n-1]$. Loop invariant after j iterations:

$$I \equiv -1 \le i < j \le n \ \wedge \ (\forall k \le i.\, a[k] \le pivot) \ \wedge \ (\forall k.\, i < k < j \Rightarrow a[k] > pivot).$$

VC2 splits on $a[j] \le pivot$: when true, incrementing i and swapping preserves both quantifier blocks; when false, j advances while i stays, and the second block extends by one. VC3 with $j = n$ yields the desired post: all elements classified. This is exactly the invariant that makes quicksort's correctness routine.

The three-region picture helps: everything left of i is $\le pivot$ (green), between $i+1$ and $j-1$ is $> pivot$ (red), and j onward is unprocessed (grey). Each iteration moves j right by one and, if needed, expands the green region. The invariant is that the coloured prefix is always correctly classified.

Example 4.1 (Pitfall: off-by-one invariant)**.** If you write I as $0 \le i \le n$ for partition where i starts at -1, VC1 fails ($i = -1 \not\ge 0$) and you chase a phantom bug. Bounds in I must match initialization. Always test VC1 on the actual initial values before proving VC2.

Remark 4.3 (Inner vs outer). For nested loops the inner invariant may temporarily break the outer one mid-iteration, but must restore it by inner exit. Think of pushing a stack: inner completes, then outer's "rows done" advances by one. If inner invariant does not imply outer's preservation, composition fails.

4.7 Exercises

E4.1 **VCs for summation.** Write VC1–3 for $s := 0$; $i := 0$; `while` $i < n$ `do` $s := s + a[i]$; $i := i + 1$ with $I \equiv 0 \le i \le n \wedge s = \sum_{j<i} a[j]$ as formal implications.

E4.2 **Search preservation.** Prove VC2 for linear search: assuming $\forall j < i.\, a[j] \neq key$ and $a[i] \neq key$, show $\forall j < i + 1.\, a[j] \neq key$.

E4.3 **Euclid exit.** From $I \equiv \gcd(a, b) = \gcd(a_0, b_0)$ and $\neg(a \neq b)$, derive $a = \gcd(a_0, b_0)$.

E4.4 **Nested sum.** Give I_{out} and I_{in} for summing an $m \times n$ matrix row-major. Explain why I_{in} must mention I_{out}.

E4.5 **Auxiliary need.** Show a program counting iterations where post mentions "number of iterations" cannot be proved without an auxiliary counter. Propose it and state its invariant.

Chapter Notes

Verification conditions originate with Floyd and King. The three-VC presentation is standard in Winskel and Nielson–Nielson. Gries gives many loop invariants; Kaldewaij's *Programming: The Derivation of Algorithms* is a rich source of nested-loop examples.

Remark 4.4 (Workflow). When stuck, write the post Q first, then derive I as "Q with n replaced by i plus bounds". For summation Q is sum to n, so I is sum to i; for search Q is $\forall j < n$, so I is $\forall j < i$. This syntactic shadowing heuristic invents I for many standard loops.

Chapter 5

Termination: Proving Programs Stop

> *"A program that never stops is correct in the most useless sense: it never gives a wrong answer because it never gives any answer."* We now prove that loops actually finish.

From zero: no termination theory assumed. We define partial vs. total correctness, introduce variants (ranking functions) and well-founded orders, and prove termination of standard loops. Lexicographic orders and non-termination counterexamples follow.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish partial and total correctness and state what total adds;

(L2) define variant and well-founded order and explain why they force termination;

(L3) prove termination of single and nested loops via numeric and lexicographic variants;

(L4) exhibit a loop that fails to terminate and diagnose why no variant exists;

(L5) state the total-correctness while-rule and apply it end-to-end.

5.1 Partial Is Not Enough

Intuition. Recall $\{P\}\,C\,\{Q\}$ means "if C terminates from P, then Q." A loop that never terminates satisfies *every* partial triple—vacuously. The canonical villain:

$$\{\text{true}\}\ \texttt{while true do skip}\ \{\text{false}\}$$

is partially correct (no terminating execution to refute false) yet clearly wrong in practice. Total correctness $[P]\,C\,[Q]$ strengthens the promise: "C *does* terminate from P and then Q holds." So total = partial + termination.

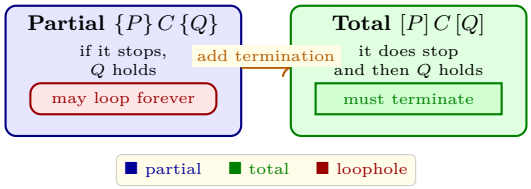

Figure 5.1: Partial correctness allows infinite executions (red strip); total correctness forbids them. Termination is the missing conjunct.

Definition 5.1 (Partial vs total)**.** $\{P\}\,C\,\{Q\}$ is *partial*: $\forall \sigma \in [\![P]\!].\ C(\sigma) \downarrow \sigma' \Rightarrow \sigma' \in [\![Q]\!]$. $[P]\,C\,[Q]$ is *total*: $\forall \sigma \in [\![P]\!].\ C(\sigma) \downarrow \sigma' \wedge \sigma' \in [\![Q]\!]$, where $\downarrow$ means terminates. For loop-free C, the two coincide.

5.2 Variants: Numbers That Must Hit Zero

Intuition. Imagine stairs labelled $5, 4, 3, 2, 1, 0$. You start at some step and promise to go down by at least one each lap, never below 0. You must hit 0 within 5 laps. A *variant* (ranking function) $V : \text{State} \to \mathbb{N}$ is that stair number: a natural that strictly decreases each iteration and is bounded below by 0. No infinite descent in $\mathbb{N}$ exists, so the loop cannot loop forever.

Definition 5.2 (Variant and well-founded)**.** A *variant* for `while` B `do` C with invariant I is an expression V such that (i) $I \wedge B \Rightarrow V \geq 0$, and (ii) $\{I \wedge B \wedge V = v_0\}\,C\,\{V < v_0\}$ where v_0 is a fresh logical variable capturing the entry value. More generally V maps to any *well-founded* order (no infinite descending chains), e.g. lexicographic pairs.

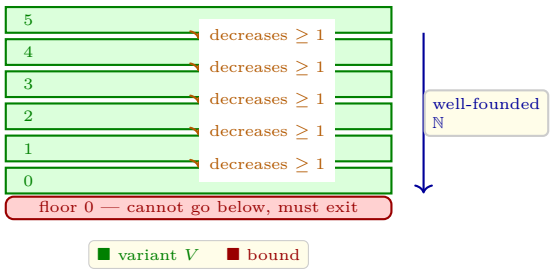

Figure 5.2: Variant as staircase: V starts at some $n \geq 0$ and drops by at least one each iteration. Hitting 0 is inevitable; infinite looping would require infinite descent in $\mathbb{N}$, which is impossible.

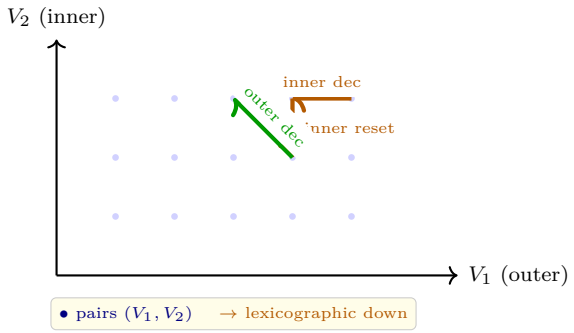

Figure 5.3: Lexicographic order (V_1, V_2): compare V_1 first, then V_2. Inner loop decreases V_2 while V_1 stays; outer decreases V_1 and resets V_2. Any lexicographic descent in $\mathbb{N} \times \mathbb{N}$ is finite.

5.3 Proving Total Correctness

Total while-rule (one form):

$$\frac{\{I \wedge B \wedge V = v_0\}\, C\, \{I \wedge V < v_0\} \qquad I \wedge B \Rightarrow V \geq 0}{\{I\}\ \texttt{while}\ B\ \texttt{do}\ C\, \{I \wedge \neg B\}}\ \text{(total)}.$$

Together with $P \Rightarrow I$ and $I \wedge \neg B \Rightarrow Q$ this gives $[P]\ \texttt{while}\ B\ \texttt{do}\ C\, [Q]$.

Example 5.1 (Countdown)**.** $[n \geq 0]\ x := n;\ \texttt{while}\ x > 0\ \texttt{do}\ x := x - 1\ [x = 0]$ with $I \equiv 0 \leq x \leq n$ and variant $V \equiv x$. Check: $I \wedge x > 0 \Rightarrow x \geq 1 \Rightarrow V \geq 0$, and $\{x = v_0\}\, x := x - 1\, \{x = v_0 - 1 < v_0\}$. So variant drops by one; termination in n steps.

Example 5.2 (Nested loops)**.** Outer $i := 0; \texttt{while}\ i < n\ \texttt{do}\ (j := 0; \texttt{while}\ j < m\ \texttt{do}\ j := j + 1;\ i := i + 1)$ has lexicographic variant $(n - i,\ m - j)$ ordered lexicographically: inner decreases second component, outer decreases first and resets second to m. No infinite lexicographic descent exists.

Example 5.3 (Euclid total)**.** Euclid loop $I \equiv a > 0 \wedge b > 0 \wedge \gcd(a, b) = \gcd(a_0, b_0)$, $V \equiv a + b$. Each iteration replaces (a, b) with $(a - b, b)$ or $(a, b - a)$, so V drops by $\min(a, b) \geq 1$ and stays ≥ 2. Hence Euclid terminates and then $a = b = \gcd(a_0, b_0)$—total correctness in one invariant plus one variant.

Listing 5.1: Variant as runtime monitor: assert decrease and boundedness each lap.

```python
def countdown(n):
    x = n
    # I: 0<=x<=n, variant V=x
    assert 0 <= x <= n
    while x > 0:
        v0 = x
        assert x >= 0          # bounded
        x -= 1
        assert x < v0          # decreased
        assert 0 <= x <= n     # invariant preserved
    assert x == 0
```

5.4 When Termination Fails

Loop $\texttt{while}\ x \neq 0\ \texttt{do}\ \ \texttt{if}\ x > 0\ \texttt{then}\ x := x - 1\ \texttt{else}\ x := x + 1$ with $I \equiv \text{true}$ and would-be variant $|x|$ does *not* decrease when $x < 0$? Actually it does ($|-2| = 2 \to |-1| = 1$). Try instead $\texttt{while}\ x > 0\ \texttt{do}\ x := x + 1$: variant x *increases*, and no $V \geq 0$ that decreases exists—the loop diverges for $x > 0$. Diagnosis: cannot find V with $I \wedge B \Rightarrow V \geq 0$ and strict decrease; the attempt to synthesize V fails, revealing the bug.

Halting in general is undecidable (Turing), so no uniform variant synthesis can succeed for all programs. For restricted loops (affine guards and updates) synthesis is decidable—this is active verification research.

Listing 5.2: A terminating and a non-terminating loop: only one admits a variant.

```javascript
// Terminates: variant n-x decreases
int x=0; while(x < n){ x++; } // V = n - x

```

```
// Diverges when n>0: no variant decreases
int y=1; while(y != 0){ y = y+1; } // overflows? unbounded increase
```

5.5 Well-Founded Orders Beyond $\mathbb{N}$

$\mathbb{N}$ suffices for many loops, but some need richer orders. Consider traversing a binary tree with a worklist: variant is multiset of remaining node heights, ordered by multiset extension of $<$—still well-founded. Another is Ackermann's function: variant is the pair (m, n) with lexicographic order on $\mathbb{N} \times \mathbb{N}$, which decreases in each recursive call even though a single number does not.

Key property: a relation $\prec$ is *well-founded* if every non-empty set has a minimal element, equivalently no infinite descending chain $x_0 \succ x_1 \succ \cdots$ exists. $\mathbb{N}$ with $<$ is well-founded; $\mathbb{Z}$ with $<$ is not (go to $-\infty$ forever); $\mathbb{N} \times \mathbb{N}$ lexicographically is well-founded (Dickson's lemma intuition). Choosing the right order is the creative step when a plain integer variant fails.

Example 5.4 (Ackermann variant)**.** Ackermann $A(m, n)$ recursion: if $m = 0$ then $n + 1$ else if $n = 0$ then $A(m - 1, 1)$ else $A(m - 1, A(m, n - 1))$. Each call reduces (m, n) lexicographically: either m drops, or m stays and n drops. Lexicographic well-foundedness guarantees termination despite the nested recursion that defeats a simple $m + n$ variant.

Remark 5.1 (Why $\mathbb{N}$ and not $\mathbb{Z}$?). $\mathbb{Z}$ is not well-founded because $\cdots 3 > 2 > 1 > 0 > -1 > -2 > \cdots$ is infinite descent. So $V : \text{State} \to \mathbb{Z}$ with only $V \geq 0$ as lower bound would not guarantee termination—you could stay ≥ 0 forever while descending $100, 99, 98, \ldots$? Actually that still terminates at 0; the danger is you could descend $0, -1, -2, \ldots$ and never hit the bound. Requiring $V \geq 0$ *and* mapped to $\mathbb{N}$ excludes negative descent.

Remark 5.2 (Invariant vs variant). Invariant answers "what stays true"; variant answers "what gets smaller". Invariant is checked once per lap (holds before and after); variant is compared across the lap ($V_{\text{after}} < V_{\text{before}}$). A loop needs both: invariant for correctness when it stops, variant to guarantee it does stop. One without the other is half a proof.

5.6 Connecting to Decidability

Turing proved halting is undecidable: no program can decide termination for all programs. So variant synthesis cannot be fully automatic. Yet for many practical loops—affine guards $a \cdot x + b > 0$ and linear updates—synthesis is decidable via linear ranking functions (Podelski–Rybalchenko). Modern verifiers try linear templates $V = c_0 + \sum c_i x_i$ and solve for c_i that satisfy the two variant conditions via linear programming. When they succeed, you get termination for free; when they fail, you must supply a lexicographic or non-linear variant manually. This interplay is why verification remains a human-machine collaboration.

5.7 Full Total-Correctness Proof: Linear Search

Claim: $[n \geq 0]\ i := 0;$ `while` $i < n \wedge a[i] \neq key$ `do` $i := i + 1\ [(i = n \Rightarrow \forall j < n.\, a[j] \neq key) \wedge (i < n \Rightarrow a[i] = key)]$.

We already have $I \equiv 0 \le i \le n \wedge \forall j < i.\, a[j] \neq key$ for partial. Add variant $V \equiv n - i$. Check: $I \wedge i < n \wedge a[i] \neq key \Rightarrow n - i > 0 \Rightarrow V \ge 0$, and body $i := i + 1$ maps $V = n - i$ to $V' = n - (i + 1) = V - 1 < V$. So total holds. Exit $I \wedge \neg(i < n \wedge a[i] \neq key)$ splits: either $i = n$ (exhausted, so $\forall j < n.\, a[j] \neq key$ from I) or $a[i] = key$ (found). No divergence case remains—variant forced exit within n steps.

This illustrates the methodology: prove partial with I, then reuse I and add V for total. The two proofs compose; you never re-prove I preservation.

5.8 Exercises

E5.1 **Countdown variant.** Prove $[n \ge 0]\, x := n; \texttt{while}\ x > 0\ \texttt{do}\ x := x - 1\, [x = 0]$ with $I \equiv 0 \le x \le n$, $V \equiv x$ by checking both variant conditions.

E5.2 **Lexicographic.** Give lexicographic variant for double loop summing $a[m][n]$ row-major and argue why single $n - i + m - j$ also works but is less modular.

E5.3 **Non-termination witness.** Give P, B, C where $\{P \wedge B\}\, C\, \{P\}$ holds (so partial proof succeeds) but no variant exists. Explain why the partial triple is vacuously useful.

E5.4 **Euclid variant.** For Euclid with gcd, propose variant $a + b$ and prove it decreases when replacing (a, b) by $(a - b, b)$ or $(a, b - a)$.

E5.5 **Total triple.** State total triple for linear search that guarantees it stops, and give invariant plus variant; show exit post now includes "found or not found" without divergence loophole.

Chapter Notes

Variants are Dijkstra's invention; well-founded orders come from set theory (Kunen). Lexicographic termination appears in Floyd and Manna. For undecidability see Sipser Ch. 4; for synthesis for linear loops see Podelski–Rybalchenko. Practice: after each invariant, immediately ask "what gets smaller?" and write V next to I.

Remark 5.3 (Variant discovery heuristic). Good V often mirrors the guard's distance to falsity: `while` $x > 0$ suggests $V = x$; `while` $i < n$ suggests $V = n - i$; `while` $a \neq b$ suggests $V = |a - b|$ or $a + b$. If guard is $x \neq y$, ask "what measure of x vs y shrinks?" The guard's shape hints at V.

Remark 5.4 (Check total early). During code review ask "does this loop have a variant?" before "is this invariant correct?" Non-termination bugs (infinite loops on edge inputs) are often cheaper to fix than postcondition bugs, and the variant question surfaces them immediately.

Chapter 6

Weakest Preconditions: Reasoning Backwards

"Program and proof develop hand in hand." Instead of guessing a precondition and checking forwards, calculate backwards: from the desired post, what must have been true before?

From zero: no wp experience assumed. We define weakest preconditions from scratch, show how substitution makes assignment trivial, and build the calculus for sequence, conditional and loops. Pictures motivate each rule before formalism.

Learning Outcomes

After this chapter you will be able to:

(L1) define $\mathrm{wp}(C, Q)$ and explain "weakest" as the largest pre-set guaranteeing Q;

(L2) compute wp for assignment, sequence, and conditional by substitution and case split;

(L3) relate wp to Hoare triples: $\{P\}\, C\, \{Q\}$ iff $P \Rightarrow \mathrm{wp}(C, Q)$;

(L4) approximate wp for loops via invariants and iterate;

(L5) use wp to generate VCs and connect to symbolic execution.

6.1 Reasoning Backwards

Intuition. Forward reasoning says: "I know P, I run C, what can I conclude?" Backward reasoning says: "I want Q afterwards, what must I ensure before?" The second is more useful for verification: the post is the goal; the pre is the obligation you must discharge at the call site.

If $\{P\}\, C\, \{Q\}$ holds, any stronger P' also works. The *weakest* precondition is the *most permissive* P that still guarantees Q—the largest set of states from which C must land in Q (if it terminates). Every other correct P

implies it.

Definition 6.1 (Weakest precondition)**.** $\mathrm{wp}(C, Q)$ is the weakest assertion such that $\{\mathrm{wp}(C, Q)\}\, C\, \{Q\}$ holds and for any P, $\{P\}\, C\, \{Q\}$ iff $P \Rightarrow \mathrm{wp}(C, Q)$. Dual: $\mathrm{sp}(P, C)$ is the strongest postcondition with $\{P\}\, C\, \{\mathrm{sp}(P, C)\}$ and $\mathrm{sp}(P, C) \Rightarrow Q$ for any valid Q.

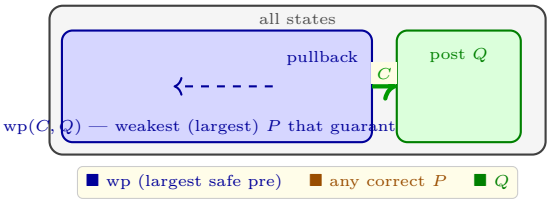

Figure 6.1: $\mathrm{wp}(C, Q)$ as the pullback of Q through C: the largest pre-set that must land in Q. Any correct P sits inside wp.

So to check $\{P\}\, C\, \{Q\}$, compute $\mathrm{wp}(C, Q)$ and test the single implication $P \Rightarrow \mathrm{wp}(C, Q)$—no need to reason about C again. This reduces verification to logic.

6.2 Calculus: wp by Syntax

Intuition. Each command transforms the desired post backwards into a pre. The rules are:

Skip: $\mathrm{wp}(\texttt{skip}, Q) = Q$—nothing changes.

Assignment: $\mathrm{wp}(x := e, Q) = Q[x \mapsto e]$—substitute e for x in Q. Example: $\mathrm{wp}(x := x + 1,\ x > 5) \equiv (x + 1 > 5) \equiv x > 4$.

Sequence: $\mathrm{wp}(C_1; C_2, Q) = \mathrm{wp}(C_1, \mathrm{wp}(C_2, Q))$—work backwards, inside out.

Conditional: $\mathrm{wp}(\texttt{if } B \texttt{ then } C_1 \texttt{ else } C_2, Q) = (B \Rightarrow \mathrm{wp}(C_1, Q)) \wedge (\neg B \Rightarrow \mathrm{wp}(C_2, Q))$, equivalently $(B \wedge \mathrm{wp}(C_1, Q)) \vee (\neg B \wedge \mathrm{wp}(C_2, Q))$ in total correctness.

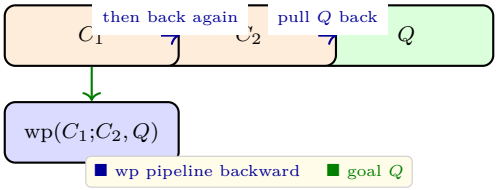

Figure 6.2: Sequence wp as a pipeline: start from Q, pull backwards through C_2, then through C_1. No forward simulation needed.

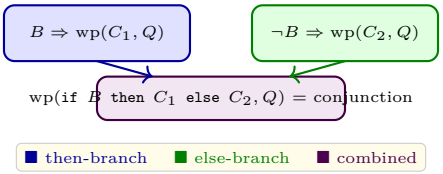

Figure 6.3: Conditional wp: guarantee Q whichever branch runs. Both implications must hold—case split on B.

Listing 6.1: Computing wp by substitution: the assignment rule as a function.

```
1 def wp_assign(var, expr_subst, Q):
```

```
    """Q is a lambda on state. Return wp(x:=e, Q) as new lambda."""
    # Q[x -> e]: evaluate Q after replacing x by e(vars)
    return lambda s: Q({**s, var: expr_subst(s)})

# Example: wp(x:=x+1, x>5) should be x>4
Q = lambda s: s['x'] > 5
wp = wp_assign('x', lambda s: s['x']+1, Q)
print(wp({'x':4})) # True? 4>4? False -> after x:=x+1, x=5 not>5, so pre false at x=4?
    check: 4+1=5 not>5 false correct
print(wp({'x':5})) # True -> 5+1=6>5 true
```

Listing 6.2: Sequence wp: pull back through C_2 then C_1.

```
/* Post Q: y == 8.  Program: x=x+1; y=2*x; */
// wp(y=2*x, y==8) = (2*x == 8) = (x==4)
// wp(x=x+1, x==4) = (x+1==4) = (x==3)
// So wp(x=x+1; y=2*x, y==8) = (x==3)
int x=3; // satisfies wp
x=x+1;   // now x==4
y=2*x;   // now y==8 == Q
```

6.3 Loops: Approximation and Invariants

Exact wp(`while` B `do` C, Q) is the limit of iterating wp for $0, 1, 2, \ldots$ unrollings—generally not first-order expressible. So we *approximate* with an invariant I:

$$\text{wp}(\texttt{while } B \texttt{ do } C, Q) \approx I \text{ where } I \wedge \neg B \Rightarrow Q \text{ and } \{I \wedge B\}\, C\, \{I\}.$$

Soundness: if you find such I and can show $P \Rightarrow I$, you have proved $\{P\}\,\texttt{while } B \texttt{ do } C\,\{Q\}$ via the while-rule; but I may not be the weakest—just a sufficient approximation. Stronger invariants give closer approximations (more of wp's power).

Iterative picture: define $F(X) = (\neg B \wedge Q) \vee (B \wedge \text{wp}(C, X))$. Then wp(`while` B `do` C, Q) $= \text{lfp}\, F$, the least fixed point. Kleene iteration $F^0(\text{false}), F^1(\text{false}), \ldots$ under-approximates; $F^0(\text{true}), F^1(\text{true}), \ldots$ over-approximates. Invariant reasoning picks a fixed point that is inductive.

Example 6.1 (Loop wp approximation)**.** $Q \equiv x = 0$, loop `while` $x > 0$ `do` $x := x - 1$. Exact wp is $x \geq 0$. With $I \equiv x \geq 0$ we capture it; with $I \equiv$ true we get Q only when $\neg B$ gives $x \leq 0$, plus I gives no lower bound—too weak to imply Q for $x = -3$. So $I =$ true is sound for partial but not precise.

Remark 6.1 (Invariant choice = wp approximation quality). A weak invariant gives a coarse over-approximation of wp (admits too many pre-states, some unsound for the goal); a strong inductive invariant gives a tighter approximation. Finding the *right* I is exactly the search for a good wp approximation. This is why invariant inference is the bottleneck of automated wp-based verification.

6.4 wp and Symbolic Execution

Modern verifiers do not ask you to write wp by hand—they compute it. Symbolic execution is wp with a symbolic store: each assignment wp becomes a substitution in the path condition. A path condition is exactly the conjunct of branch decisions plus the pulled-back post. Feeding $\neg(P \Rightarrow \mathrm{wp}(C, Q))$ to an SMT solver searches for a counterexample state satisfying P but refuting Q after C.

Remark 6.2 (Automation). Dafny, Why3, and Boogie all generate wp-style VCs. You write $\{P\}\, C\, \{Q\}$ with loop invariants; the tool emits $P \Rightarrow \mathrm{wp}(C, Q)$ and asks Z3 to prove it. Understanding wp tells you why missing invariants make the solver complain "cannot prove" and how the counterexample maps to a concrete trace.

6.5 Dual: Strongest Postconditions

Dually, $\mathrm{sp}(P, C)$ pushes P forward. Rules: $\mathrm{sp}(P, \texttt{skip}) = P$, $\mathrm{sp}(P, x := e) = \exists x_0.\, P[x_0/x] \wedge x = e[x_0/x]$, $\mathrm{sp}(P, C_1; C_2) = \mathrm{sp}(\mathrm{sp}(P, C_1), C_2)$, $\mathrm{sp}(P, \texttt{if } B \texttt{ then } C_1 \texttt{ else } C_2) = \mathrm{sp}(P \wedge B, C_1) \vee \mathrm{sp}(P \wedge \neg B, C_2)$. Intuition: assignment introduces an existential over the old value; conditionals become disjunction of the two pushed branches.

Forward vs. backward: sp mirrors how testing runs code forward, accumulating facts; wp mirrors how proofs propagate goals backward. Verifiers historically chose wp because it avoids existentials—substitution is cheaper for SMT than $\exists$. Symbolic execution is sp-style (path condition accumulates forward), but VC generation is wp-style (goal propagates backward); the two meet in the middle at the program midpoint.

Example 6.2 (sp vs wp)**.** $P \equiv x = 3$, $C \equiv x := x + 1$. $\mathrm{wp}(C, x = 4) \equiv x = 3 = P$, so P is exactly weakest. $\mathrm{sp}(P, C) \equiv \exists x_0.\, x_0 = 3 \wedge x = x_0 + 1 \equiv x = 4$. Here both sides coincide; for $Q \equiv x > 0$, $\mathrm{wp}(C, Q) \equiv x > -1$ is weaker than P, showing many P map to same post forward, but only one weakest backward.

6.6 Worked End-to-End: Swap

Prove $\{x = a \wedge y = b\}\ t := x;\ x := y;\ y := t\ \{x = b \wedge y = a\}$ via wp.

Compute backward:

$$\mathrm{wp}(y := t,\ x = b \wedge y = a) \equiv x = b \wedge t = a,$$
$$\mathrm{wp}(x := y,\ x = b \wedge t = a) \equiv y = b \wedge t = a,$$
$$\mathrm{wp}(t := x,\ y = b \wedge t = a) \equiv y = b \wedge x = a.$$

So $\mathrm{wp}(C, Q) \equiv x = a \wedge y = b$, which is exactly P; thus $P \Rightarrow \mathrm{wp}(C, Q)$ holds with equality. No invariant needed (loop-free). This calculational style—repeated substitution, no forward guessing—is the hallmark of wp proofs. For programs with conditionals, the conjunction of branch cases replaces the single substitution but the backward pipeline remains.

Remark 6.3 (Calculational vs axiomatic). Hoare logic checks a proof tree you supply; wp calculates the obligation you must check. The two are equivalent (soundness/completeness), but wp is more mechanical: compute, then ask the solver. This is why automated verifiers prefer wp generation over searching for Hoare derivations.

6.7 Exercises

E6.1 **Assignment.** Compute $\mathrm{wp}(x := 2 * x + 1,\ x > 10)$ and $\mathrm{wp}(t := x;\ x := y;\ y := t,\ x = a \wedge y = b)$ (swap).

E6.2 **Sequence.** Compute $\mathrm{wp}(x := x + 1;\ y := x * 2,\ y > 6)$. Show intermediate $\mathrm{wp}(y := x * 2,\ y > 6)$ then pull through $x := x + 1$.

E6.3 **Conditional.** Compute $\mathrm{wp}(\texttt{if}\ x > 0\ \texttt{then}\ x := x - 1\ \texttt{else}\ x := 0,\ x \geq 0)$. Simplify the conjunction.

E6.4 **Loop approximation.** For $\texttt{while}\ x > 0\ \texttt{do}\ x := x - 1$ with $Q \equiv x = 0$, compare $I \equiv x \geq 0$ vs $I \equiv \text{true}$ in precision and show where true fails to prove $P \equiv x = 2 \Rightarrow Q$.

E6.5 **Hoare vs wp.** Prove $\{P\}\, C\, \{Q\}$ iff $P \Rightarrow \mathrm{wp}(C, Q)$ for loop-free C by induction on C.

Chapter Notes

wp is Dijkstra (1975, *Guarded Commands*). Gries (1981) and Dijkstra–Scholten (1990) are the classic calculational texts. Winskel and Pierce's *Software Foundations* connect wp to Hoare logic formally. For symbolic execution see King (1976) and modern KLEE/Dafny papers.

Remark 6.4 (Reading tip). When you see $\mathrm{wp}(x := e, Q) = Q[x \mapsto e]$, read '$[x \mapsto e]$' as "what Q says about the *new* x must have been true about e in the *old* state." Substitution is time travel: it translates a claim about the future into a claim about the present.

Chapter 7

Separation Logic: Reasoning About Heap and Pointers

"A program without pointers is a house without rooms; a logic without $*$ is a map without walls." Separation logic adds the walls back so you can reason about one room at a time.

From zero: no separation logic assumed. We start from why ordinary Hoare logic stumbles on aliasing, build the heap model, introduce separating conjunction and its rules, and prove tiny heap programs. Only basic pointers and Hoare triples are assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) explain why aliasing breaks the Hoare assignment axiom;

(L2) define heap, heaplet, and the satisfaction of points-to $x \mapsto v$;

(L3) use separating conjunction $*$ and emp to describe disjoint heaps;

(L4) apply the heap axioms (lookup, mutate, new, dispose) and the frame rule;

(L5) specify list segments and prove in-place list reversal sketch.

7.1 Why Hoare Logic Stumbles on the Heap

Intuition. Ordinary assertions like $x = 5$ describe the *store* (variables). They have no language for "the heap cell at address x holds 5". Worse, aliasing means two names can denote the same cell: after $y := x$, writing through y changes what you read through x. The Hoare assignment axiom $Q[x \mapsto e]$ assumes variables are distinct names for distinct slots; for heap writes $[x] := e$ that assumption is false when x and y alias.

Example 7.1 (Aliasing counterexample)**.**

$$\{x \mapsto 0 * y \mapsto 0\}\ [x] := 3\ \{x \mapsto 3 * y \mapsto 0\}$$

is claimed. But if $x = y$ (same address), after $[x] := 3$ we have $x \mapsto 3 * y \mapsto 3$ (they are the same cell!), so the post claiming $y \mapsto 0$ is false. Ordinary conjunction $x \mapsto 0 \wedge y \mapsto 0$ does not preclude $x = y$; we need a connective that *forces* disjointness. That is $*$.

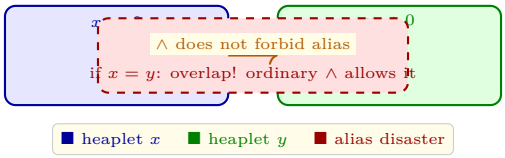

Figure 7.1: Ordinary $\wedge$ allows x and y to alias the same cell (red overlap). $x \mapsto 0 \wedge y \mapsto 0$ is true even when $x = y$, so mutating $[x]$ silently mutates y.

Separation logic fixes this by splitting the heap into *disjoint heaplets*. $P * Q$ means: the heap can be split into two disjoint parts, one satisfying P, the other Q. So $x \mapsto 0 * y \mapsto 0$ *guarantees* $x \neq y$—there would be no way to split a single cell to satisfy both. Mutation of x cannot affect y.

7.2 Heaps, Points-To, and Separating Conjunction

Definition 7.1 (Heap)**.** A *heap* h is a finite partial map from addresses (values) to values. Write $dom(h)$ for its domain. Two heaps h_1, h_2 are *disjoint* if $dom(h_1) \cap dom(h_2) = \emptyset$; then $h_1 \uplus h_2$ is their union.

Definition 7.2 (Assertions)**.** emp holds only on empty heap. $e_1 \mapsto e_2$ holds on heap with exactly one cell at address e_1 holding e_2. $P * Q$ holds on h iff $\exists h_1, h_2.\, h = h_1 \uplus h_2 \wedge h_1 \models P \wedge h_2 \models Q$. $P \wedge Q$ shares the *same* heap; $P * Q$ *splits* it. $P -\!\!* Q$ (magic wand) is the adjunct: $h \models P -\!\!* Q$ iff for all h' disjoint from h, $h' \models P \Rightarrow h \uplus h' \models Q$.

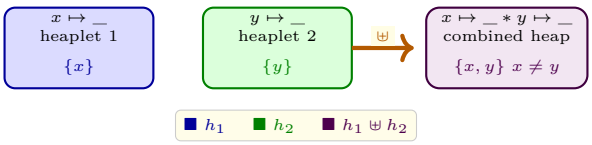

Figure 7.2: Separating conjunction $*$ as disjoint union: $x \mapsto \_$ and $y \mapsto \_$ occupy separate cells, so $x \mapsto \_ * y \mapsto \_$ forces $x \neq y$. The heap is the sum of its heaplets.

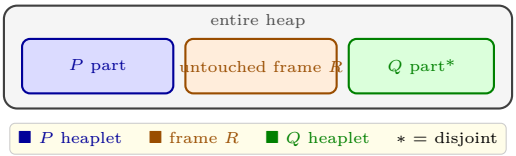

Figure 7.3: Frame intuition: prove $\{P\}\,C\,\{Q\}$ on its footprint; any disjoint frame R stays untouched, so $\{P * R\}\,C\,\{Q * R\}$ holds. $*$ makes local reasoning sound.

Example 7.2 (* vs $\wedge$)**.** $x \mapsto 3 * y \mapsto 4$ implies $x \neq y$ and heap has exactly two cells. $x \mapsto 3 \wedge y \mapsto 4$ on a two-cell heap with $x \neq y$ also holds, but on a one-cell heap with $x = y = 5$ and content 3 (and 4? impossible—one cell cannot be both 3 and 4, so $\wedge$ still fails for that reason). The key difference: $x \mapsto \_ * y \mapsto \_$ *forces* two cells;

$x \mapsto \_ \wedge y \mapsto \_$ does not force disjointness in the logic, but for singletons both happen to force distinctness because one cell cannot satisfy both points-to simultaneously unless contents equal.

7.3 Heap Axioms and the Frame Rule

Heap commands (with store x, y and heap h):

Lookup: $\{x \mapsto \_\}\ y := [x]\ \{y = v \wedge x \mapsto v\}$ where v is the old content.

Mutate: $\{x \mapsto \_\}\ [x] := e\ \{x \mapsto e\}$.

New: $\{\text{emp}\}\ x := \text{new}(e)\ \{x \mapsto e\}$ (allocates fresh address).

Dispose: $\{x \mapsto \_\}\ \text{dispose}(x)\ \{\text{emp}\}$.

The star of the show is the *frame rule*:

$$\frac{\{P\}\, C\, \{Q\}}{\{P * R\}\, C\, \{Q * R\}} \text{ provided } C \text{ does not modify free vars of } R.$$

If C is correct on its footprint P, it remains correct in any larger heap that adds a disjoint R—because C never touches outside its footprint. This is *local reasoning*: prove the small heap, get the big heap for free.

Listing 7.1: Heap axioms in C-like syntax: lookup, mutate, alloc. $*$ keeps footprints disjoint.

```
// { x |-> _ }   x points somewhere
y = *x;          // { x |-> v && y==v }  y gets heap content v
// { x |-> _ }
*x = 3;          // { x |-> 3 }          heap cell updated
// { emp }
x = malloc(1); *x = 5; // { x |-> 5 }  fresh cell
free(x);         // { emp }              back to empty
```

Listing 7.2: Modelling $*$ as disjoint dict union: $P * Q$ holds iff heap splits.

```
def sep_conj(P, Q, heap):
    """Check P*Q on heap (dict addr->val) by searching split."""
    addrs = list(heap.keys())
    from itertools import combinations
    for r in range(len(addrs)+1):
        for left_addrs in combinations(addrs, r):
            h1 = {a:heap[a] for a in left_addrs}
            h2 = {a:heap[a] for a in addrs if a not in left_addrs}
            if P(h1) and Q(h2):
                return True
    return False
# P = "x|->0": heap {x:0}; Q = "y|->0": heap {y:0}
# P*Q needs heap {x:0,y:0} with x!=y
```

7.4 Lists and Local Reversal

Define $\mathrm{ls}(x, y)$: linked list segment from x to y (where y is null or another pointer). Inductive: $\mathrm{ls}(x, x) \equiv \mathrm{emp}$ (empty) and $\mathrm{ls}(x, y) \equiv \exists n.\, x \mapsto n * \mathrm{ls}(n, y)$ when $x \neq y$. Then full list $\mathrm{list}(x) \equiv \mathrm{ls}(x, \mathrm{null})$.

Reversal sketch: $\{\mathrm{list}(p)\}\ r := \mathrm{null}; \texttt{while}\ p \neq \mathrm{null}\ \texttt{do}\ (n := [p+1]; [p+1] := r; r := p; p := n)\ \{\mathrm{list}(r) \wedge \mathrm{list}(p) \equiv \text{original reversed}\}$. Invariant: $\mathrm{ls}(r, \mathrm{null}) * \mathrm{ls}(p, \mathrm{null})$ where the two segments are exactly the reversed prefix and the unprocessed suffix, and together they partition the original heap. Frame rule lets each iteration reason only about $p \mapsto n$ and r's head.

Example 7.3 (Two heaplets, one invariant)**.** Before iteration: heap $= \underbrace{r \mapsto \cdots * \cdots}_{reversed} * \underbrace{p \mapsto n * \mathrm{ls}(n, \mathrm{null})}_{suffix}$. Iteration swings p's next to r, so new $r' = p$ and new $p' = n$. The invariant re-establishes as $\mathrm{ls}(p', \mathrm{null})$ is the tail and $\mathrm{ls}(r', \mathrm{null})$ is the grown reversed prefix. $*$ guarantees the prefix and suffix never share a cell, so the swing cannot create a cycle.

7.5 Magic Wand and Iterated Star

$P - \!\!* Q$ is for "if you give me a P-heap, together we make Q". It is the right adjoint to $*$: $(P * Q \Rightarrow R)$ iff $(P \Rightarrow Q - \!\!* R)$. Intuition: $Q - \!\!* R$ is the "heap remainder" needed to go from Q to R. Example: $(x \mapsto \_) - \!\!* (x \mapsto 3)$ is the claim "adding a heaplet where x holds whatever is required to make $x \mapsto 3$"—essentially an update promise.

Iterated $*$ describes arrays: $\circledast_{i=0}^{n-1} a + i \mapsto \_$ means n disjoint cells at $a, \ldots, a+n-1$. This lets specs of `memcpy` stay local: $\{\circledast_{i<n} src + i \mapsto v_i * \circledast_{i<n} dst + i \mapsto \_\}\ \texttt{memcpy}(dst, src, n)\ \{\circledast_{i<n} src + i \mapsto v_i * \circledast_{i<n} dst + i \mapsto v_i\}$. Without $*$, aliasing between src and dst would make the spec unsound; with $*$, $src \neq dst$ and non-overlapping are enforced structurally.

Remark 7.1 ($*$ owns, $\wedge$ shares). Think $*$ as "owns disjointly" and $\wedge$ as "shares aliased view". Ownership is what makes frame sound: you can frame only what you do not own in P. This is why Rust's borrow checker and separation logic's $*$ feel similar—both track disjoint ownership.

7.6 Why Frame Prevents Leaks and Double-Free

Separating conjunction also tracks *resource accounting*. $x \mapsto \_ * y \mapsto \_$ says there are exactly two cells owned; after dispose(x) you own $y \mapsto \_$ alone—x's cell is gone. Double-free $\{x \mapsto \_\}$ dispose(x); dispose(x) fails: after first dispose you own emp, so second dispose's pre $x \mapsto \_$ cannot be satisfied—no frame can rescue it because $\mathrm{emp} * x \mapsto \_$ would require a fresh x cell you do not own.

Leak detection is dual: $\{x \mapsto 3\}\ \texttt{skip}\ \{\mathrm{emp}\}$ is unprovable because emp demands no cells but you still own $x \mapsto 3$; no $*$ manipulation discards owned heap. This is how separation logic verifiers (Infer) catch leaks at compile time: owned heap that is not freed is a $*$ remainder that never reaches emp.

Remark 7.2 (Garbage collection). In GC languages, dispose is implicit and the post of dropping a reference is not emp but a weaker "garbage may remain". Separation logic adapts by treating GC as an implicit frame: collected cells move to an invisible R that the spec need not mention. The $*$ still forces disjointness before collection.

7.7 Exercises

E7.1 **Alias vs $*$.** Show $x \mapsto 0 * y \mapsto 0 \Rightarrow x \neq y$ but $x \mapsto 0 \wedge y \mapsto 0$ does not imply $x \neq y$ on a heap where maps are sets? Explain via $*$'s split definition.

E7.2 **Mutate with frame.** From $\{x \mapsto \_ * R\}\,[x] := 3\,\{x \mapsto 3 * R\}$ derive why R is untouched; give $R \equiv y \mapsto 4$ and show the post still claims $y \mapsto 4$.

E7.3 **List emp base.** Prove $\mathrm{ls}(x, x) \equiv \mathrm{emp}$ and $\mathrm{ls}(x, y) * \mathrm{ls}(y, z) \Rightarrow \mathrm{ls}(x, z)$ by induction on length.

E7.4 **Dispose spec.** State $\{x \mapsto v * R\}\,\mathrm{dispose}(x)\,\{R\}$ using frame; why is $*$ needed rather than $\wedge$?

E7.5 **Reversal invariant picture.** Draw heaplets before and after one iteration of in-place reversal, labelling p, r, n and shading $*$ footprints blue/green with untouched suffix grey.

Chapter Notes

Reynolds (2002) and O'Hearn–Pym introduced separation logic; "Separation Logic: A Logic for Shared Mutable Data Structures" is the classic. For lists see Reynolds Ch. 1–2, Yang and Birkedal notes. Frame rule as local reasoning is O'Hearn's key insight; modern tools are Infer, VeriFast, and Viper.

Remark 7.3 (Scaling to concurrency). The same $*$ that gives frame for sequential heaps gives *concurrent* separation logic's resource sharing: $P * Q$ can be split between threads with no race on owned cells. This is why separation logic became the foundation for verified concurrent data structures—ownership as logic. See O'Hearn's Concurrent Separation Logic (2007) for the direct extension.

Remark 7.4 (Mental model). Draw every heap assertion as coloured boxes that cannot overlap if joined by $*$. If you can shade the heaplet of each $*$ conjunct with a distinct colour without overlap, the assertion is plausible; if two conjuncts demand the same cell with different contents, it is false.

Chapter 8

Refinement and Correctness by Construction

"Do not verify after the fact; construct so that correctness is the only outcome." Refinement turns a specification into code by small, proved steps—like carving a statue by successive, measured cuts.

From zero: no refinement background assumed. We define refinement from specifications to programs, present Morgan's laws, and build two programs—binary search and list reversal—by calculation, not guess-and-verify. Only Hoare triples and invariants are assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define refinement $S \sqsubseteq C$ and its relation to Hoare triples;

(L2) apply refinement laws for composition, conditional, and loop introduction;

(L3) derive a loop from a spec via invariant and variant as construction steps;

(L4) contrast post-hoc verification with correctness by construction;

(L5) sketch a refinement chain from abstract spec to executable heap code.

8.1 From Post-Hoc Proof to Construction

Intuition. Post-hoc verification writes code first, then tries to prove $\{P\}\, C\, \{Q\}$. When the proof fails you patch code and retry—a costly loop. Correctness by construction inverts the flow: start from the spec $S \equiv \{P\}\, ?\, \{Q\}$ (a hole where code will go) and *refine* it stepwise, each step preserving correctness. When the hole is filled, no separate proof is needed—the construction *is* the proof.

Definition 8.1 (Specification statement and refinement)**.** Write $x : [P, Q]$ for "change only x to establish Q,

assuming P". More generally $w : [P, Q]$ where w is the frame (variables allowed to change). Then $S \sqsubseteq C$ ("C refines S") iff for all P, Q with $\{P\}\, S\, \{Q\}$, also $\{P\}\, C\, \{Q\}$—every spec satisfied by S is satisfied by C. In Hoare terms, $w : [P, Q] \sqsubseteq C$ iff $\{P\}\, C\, \{Q\}$ and C modifies only w.

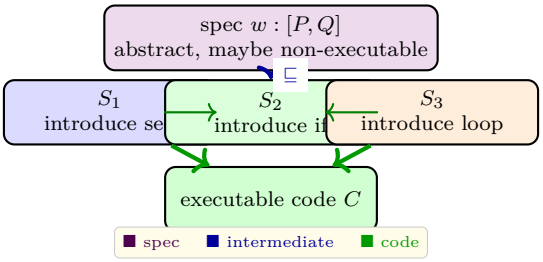

Figure 8.1: Refinement lattice: start at spec (top, violet), descend by law-driven steps to code (bottom, green). Each arrow $\sqsubseteq$ is a proved law, so correctness flows downward.

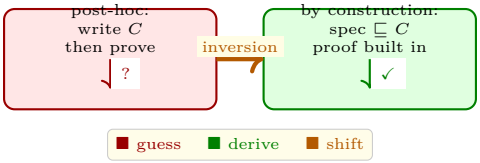

Figure 8.2: Two workflows: post-hoc guesses code then struggles to prove; construction derives code from spec so the proof is intrinsic. The arrow is inverted.

8.2 Laws of Refinement

Each law is an equivalence justified by Hoare logic. Core laws (Morgan):

Skip: $w : [P, P] \sqsubseteq$ `skip`.

Assignment: $x : [P, Q[x \mapsto e]] \sqsubseteq x := e$ when $x \in w$.

Sequence: $w : [P, R] \sqsubseteq w : [P, Q]; w : [Q, R]$ (introduce intermediate Q).

Conditional: $w : [P, Q] \sqsubseteq$ `if` B `then` $w : [P \wedge B, Q]$ `else` $w : [P \wedge \neg B, Q]$.

Loop (with invariant I and variant V): $w : [I,\ I \wedge \neg B] \sqsubseteq$ `while` B `do` $w : [I \wedge B,\ I \wedge V < v_0]$ where v_0 is logical snapshot of V on entry.

Each law's side conditions are VCs; discharging them guarantees the step is sound. So refinement is Hoare logic turned into a *design* calculus.

Example 8.1 (Conditional refinement)**.** Spec to compute $m = \max(a, b)$: $m : [\text{true},\ m \geq a \wedge m \geq b \wedge (m = a \vee m = b)]$. By conditional law with $B \equiv a \geq b$, refine to `if` $a \geq b$ `then` $m : [a \geq b,\ m = a]$ `else` $m : [a < b,\ m = b]$, then each branch refines by assignment $m := a$ and $m := b$ respectively. The B case split was forced by the spec's disjunction.

Remark 8.1 (Data refinement). Beyond code, you refine *data*: abstract set S to concrete array a with coupling invariant $S = \{a[i] : 0 \leq i < n\}$. Operations on S (add, member) refine to array manipulations that preserve the coupling. This is how abstract specs of collections become efficient implementations without breaking proved properties.

Listing 8.1: Specification statement as JML contract: the abstract spec to be refined.

```
/* @ requires n >= 0 && sorted(a,0,n-1);
  @ ensures (\result==-1 ==> forall k. a[k]!=key) &&
  @         (\result>=0 ==> a[\result]==key);
  @ assignable \nothing; // abstract spec, no code yet
  @*/
// w:[P,Q] where w={result}, P=sorted&&n>=0, Q=found-or-not
int spec_search(int[] a,int n,int key);
```

8.3 Construction in Action: Binary Search

Spec: $r : [\text{sorted}(a[0..n)) \wedge 0 \le n,\ (r \ge 0 \Rightarrow a[r] = key) \wedge (r = -1 \Rightarrow key \notin a)]$.

Step 1—introduce variables lo, hi and strengthen invariant:

$$I \equiv 0 \le lo \le hi \le n \wedge (\forall i < lo.\, a[i] < key) \wedge (\forall i \ge hi.\, a[i] > key)$$

with $lo := 0; hi := n$ establishing I (VC: $P \Rightarrow I[lo, hi \mapsto 0, n]$).

Step 2—loop `while` $lo < hi$ `do` with variant $V \equiv hi - lo$. Introduce mid $m := (lo + hi)/2$ inside, then conditional on $a[m]$ vs. key: if $a[m] < key$ then $lo := m+1$ else if $a[m] > key$ then $hi := m$ else $r := m$; $lo := hi$ (found). Each branch preserves I and decreases V (gap shrinks by at least half). Exit $lo = hi$ gives $I \wedge lo = hi \wedge \neg(lo < hi)$ plus the two quantified halves imply key absent, so $r := -1$ establishes Q.

Calculation produced the classic binary search without ever running a test—the invariant dictated every branch.

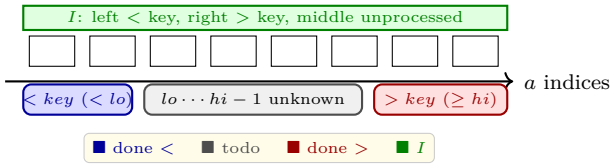

Figure 8.3: Binary search invariant as three regions: processed $<$ key (blue), unprocessed $[lo, hi)$ (grey), processed $>$ key (red). Each iteration shrinks grey and preserves colours.

Listing 8.2: Calculated binary search: each assignment justified by invariant preservation.

```
def bsearch_calc(a, key):
    lo, hi = 0, len(a)
    # I: all <lo are <key, all >=hi are >key (sorted)
    while lo < hi:
        m = (lo + hi)//2
        if a[m] < key: lo = m + 1    # preserves I, V decreases
        elif a[m] > key: hi = m      # preserves I
        else: return m               # found
    return -1  # I and lo==hi => key not in a
```

8.4 Heap Refinement: List Reversal Redux

Abstract spec: $r : [\text{list}(p),\ \text{list}(r) \wedge \text{rev}(r) = \text{rev}_0(p)]$ where rev_0 is initial list. Refinement introduces $r := \text{null}$ then loop with separation invariant $\text{ls}(r, \text{null}) * \text{ls}(p, \text{null})$ and $\text{rev}(r) +\!\!+ \text{rev}(p) = \text{rev}_0$. Loop body $n := [p + 1]; [p + 1] := r; r := p; p := n$ is exactly the four-command sequence that $*$ proves local: it touches only $p \mapsto n$, so frame $R \equiv \text{ls}(r, \text{null}) * \text{ls}(n, \text{null})$ is preserved. This is Chapter 7's reversal now *derived*, not verified post-hoc.

8.5 Mini-Derivation: Summation Lawfully

Derive $s := [n \geq 0,\ s = \sum_{j=0}^{n-1} a[j]]$ where s is result frame.

Start: $s : [n \geq 0,\ s = \text{sum}]$. By sequence law with $Q \equiv s = 0 \wedge i = 0$, refine to $s, i : [n \geq 0, I];\ s, i : [I,\ I \wedge i = n]$ where $I \equiv 0 \leq i \leq n \wedge s = \sum_{j<i} a[j]$. First half refines to $s := 0; i := 0$ by assignments (init VC).

Second half, loop law with $B \equiv i < n$, I as invariant, $V \equiv n - i$: refine to `while` $i < n$ `do` $s, i : [I \wedge i < n,\ I \wedge V < v_0]$. Body $s, i : [I \wedge i < n,\ I \wedge V < v_0]$ refines to $s := s + a[i]; i := i + 1$ (preserve VC plus $n - (i + 1) < n - i$). Exit $I \wedge i \geq n \Rightarrow I \wedge i = n \Rightarrow s = \text{sum}$ discharges. At no point did we guess the loop—invariant I dictated its shape, variant V its termination.

Remark 8.2 (Derivation as search). Refinement laws are non-deterministic: you choose Q, I, V. Failed choice surfaces as an unprovable VC, guiding correction. So derivation is *proof-directed search* with immediate feedback, not blind guessing. This is why beginners who write code first struggle: they search in code space without VC feedback.

Remark 8.3 (Reuse via refinement). Once $w : [P, Q] \sqsubseteq C$ is proved, C can replace the spec everywhere the spec appears—no re-proof needed at call sites. This is modularity as logic: specs are interfaces, refinements are verified implementations. Libraries shipped as specs plus proved code compose without re-verification, just frame.

8.6 When to Refine vs Verify

Use *verify* when code exists and is small enough to annotate (legacy code, library function). Use *refine* when building new critical code (OS kernel, crypto) where cost of bug dwarfs cost of derivation. Hybrid is common: refine the skeleton, verify leaf procedures. Tools support both: Dafny's `method` is a spec to refine, its body is verified post-hoc against the spec you calculated. In practice the line blurs—refinement *produces* the annotations that make verification automatic.

Example 8.2 (Choosing)**.** Sorting routine for a one-off script: verify post-hoc (quick tests plus light invariants). Sorting inside a pacemaker's firmware: derive via refinement, with I capturing "prefix sorted and bounded by suffix", variant $n - i$, and $*$ tracking buffer ownership. The extra derivation effort is insurance.

8.7 Exercises

E8.1 **Refine skip.** Prove $x : [P, P] \sqsubseteq$ `skip` using the Hoare skip axiom. Why is the frame condition needed?

E8.2 **Sequence law.** From $w : [P, R] \sqsubseteq w : [P, Q]; w : [Q, R]$, show how choosing Q as invariant splits a spec into two provable halves.

E8.3 **Binary search VC.** Discharge preservation for the branch $a[m] < key \Rightarrow lo := m + 1$ with I as above; show V strictly decreases.

E8.4 **Heap frame in refinement.** For list reversal, state the frame R for body $n := [p + 1]; [p + 1] := r$ and show $\{p \mapsto n * R\}\, C\, \{p \mapsto r * R\}$ fits the frame rule.

E8.5 **Construction vs verification.** Give a tiny program where post-hoc proof needs an auxiliary variable but calculational refinement avoids it by introducing the variable at spec level. Explain.

Chapter Notes

Refinement calculus is Back (1978), Morgan (1990, *Programming from Specifications*), Morris (1987), and Back–von Wright. Dijkstra–Scholten links wp to refinement. Binary search derivation is classic in Gries and Kaldewaij. For heap refinement see Gardner et al. and the VeriFast approach: $*$ plus refinement scales to verified OS kernels (seL4).

Remark 8.4 (Next steps). You now have the full stack: specs as contracts (Ch 2), invariants for loops (Ch 3–5), backward calculation via wp (Ch 6), heap ownership via $*$ (Ch 7), and stepwise descent to code (Ch 8). A real project chains them: spec $w : [P, Q] \xrightarrow{\text{wp}+I+V}$ loop $\xrightarrow{*+\text{frame}}$ heap code $\xrightarrow{\text{SMT}}$ checked. That chain is what Dafny, Why3, and Viper automate—your job is to supply the creative I, V, and $*$ footprints.